

EventsApp XSS Mitigation & Data Sanitization Milestone 3

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Objective

Harden the `coding/CST-407-RS-T5-Milestone-Eventsapp` project against cross-site scripting by identifying the vulnerable rendering paths, sanitizing user-controlled content before it hits the UI, and validating that payloads now render as harmless text while preserving the underlying data for auditing.

Video Demonstration

Link to 5-minute demonstration video: <https://youtu.be/zql0zNkvO94>

The recording walks through (1) the vulnerable Thymeleaf views and the stored XSS proof-of-concept, (2) the Sanitization Service and repository refactor that now escape user input and accept special characters in SQL statements, and (3) the live UI tests showing payloads such as `<script>alert('demo')</script>` rendering visibly as text with no execution.

1. Identify XSS Vulnerable Points

Rendering Without Escaping

- `eventsapp/src/main/resources/templates/events.html` originally used unescaped expressions (`th:utext`) in the Events table and in the `message` banner. Any stored value in `name`, `location`, or `description` was injected directly into the DOM.
- `EventController` (`eventsapp/src/main/java/com/shadsluiter/eventsapp/controllers/EventController.java`) passed raw `EventModel` objects and the raw search query straight to the view.

Demonstrated Attack

- Log in and create an event with `<script>alert('StoredXSS')</script>` in the description.
- Return to `/events`: the browser immediately executes the alert because the description cell renders the string verbatim.
- Submit `<img src=x onerror=alert('ReflectedXSS')>` on `/events/search`; the results banner runs the payload instantly.

These two flows satisfy the assignment requirement to prove the flaw and show impact.

2. Implement Data Sanitization

Sanitization Service

- Added `eventsapp/src/main/java/com/shadsluiter/eventsapp/service/SanitizationService.java`, which clones an `EventModel` and escapes every user-controlled field via `HtmlUtils.htmlEscape`.
- Provides helpers to sanitize both single events and collection views, plus a `sanitizeText` utility for ad-hoc strings (e.g., search banners).

Controller Integration

- `EventController.getAllEvents` and `EventController.search` now call `sanitizeForDisplay` before binding `events` or `message` attributes. This ensures any string printed with `th:utext` is pre-escaped.

Template Adjustments

- Because the data reaches the view already escaped, `events.html` safely uses `th:utext` for the banner and each column. This lets instructors view the literal payloads (e.g., `<script>alert('demo')</script>`) as proof that attempts are neutralized.

Repository Hardening

- `eventsapp/src/main/java/com/shadsluiter/eventsapp/data/EventRepository.java` now uses prepared statements, a `GeneratedKeyHolder`, and parameterized LIKE clauses. This eliminates SQL errors when payloads contain quotes and closes SQL injection gaps.

3. Test the Application

Manual Verification

Scenario	Payload	Expected Result	Observed Result
Create event (<code>name</code> , <code>location</code> , <code>description</code>)	<code><script>alert('NAME')</script></code> , <code></code> , <code><svg onload=alert('DESC')></svg></code>	Rows display literal payloads; no dialogs; Edit form still shows raw input	✅ Payloads appear as text, no execution, raw values persist
Search banner	<code><script>alert('SEARCH')</script></code>	Banner shows escaped string; no execution	✅ Banner prints <code><script>...</code> <code></script></code> as text
API round-trip (<code>POST /api/events</code> + <code>GET /api/events</code>)	JSON fields containing <code><script></code>	GET response includes escaped strings	✅ Sanitized output confirmed

Automated Tests

- `cd coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp && mvn -q test`
- Ensures Spring Boot context + security stack continue to load with the new service and repository changes.

Source Code Changes

- `eventsapp/src/main/resources/templates/events.html` – render messages and event columns with `th:utext` so sanitized output displays exactly what users entered.
- `eventsapp/src/main/java/com/shadsluiter/eventsapp/service/SanitizationService.java` – new service providing HTML escaping for individual fields, models, and collections.
- `eventsapp/src/main/java/com/shadsluiter/eventsapp/controllers/EventController.java` – sanitize event collections and search banner text before rendering.
- `eventsapp/src/main/java/com/shadsluiter/eventsapp/data/EventRepository.java` – parameterized all SQL statements and added generated key handling to accept payloads with quotes safely.
- `eventsapp/src/main/java/com/shadsluiter/eventsapp/service/EventService.java` – reverted to returning raw `EventModel` objects so sanitization happens exclusively at the presentation layer.